

C++ PROGRAMMING SERIES

OBJECT STREAMS SERIALIZATION IN C++

Boost.Serialization · Custom Stream Operators · Object Graphs · Real-World Systems

Stream Concepts

operator<< >>

Stream files/
formatting

File Streams for
Objects

String Streams

Real Projects

TABLE OF CONTENTS

01

What Are Object Streams?

Stream abstraction, C++ I/O model, stream hierarchy

03

Stream Flags & Formatting

ios flags, manipulators, formatting state machines

05

String Streams (stringstream)

ostringstream, istream, round-trip serialization

07

Best Practices & Summary

Security, performance, pitfalls, format selection

02

Overloading operator<< and >>

Custom stream insertion and extraction operators

04

File Streams for Objects

fstream, ifstream, ofstream with custom types

06

Real-World Projects

Student DB, config, game save, network objects

SECTION 01

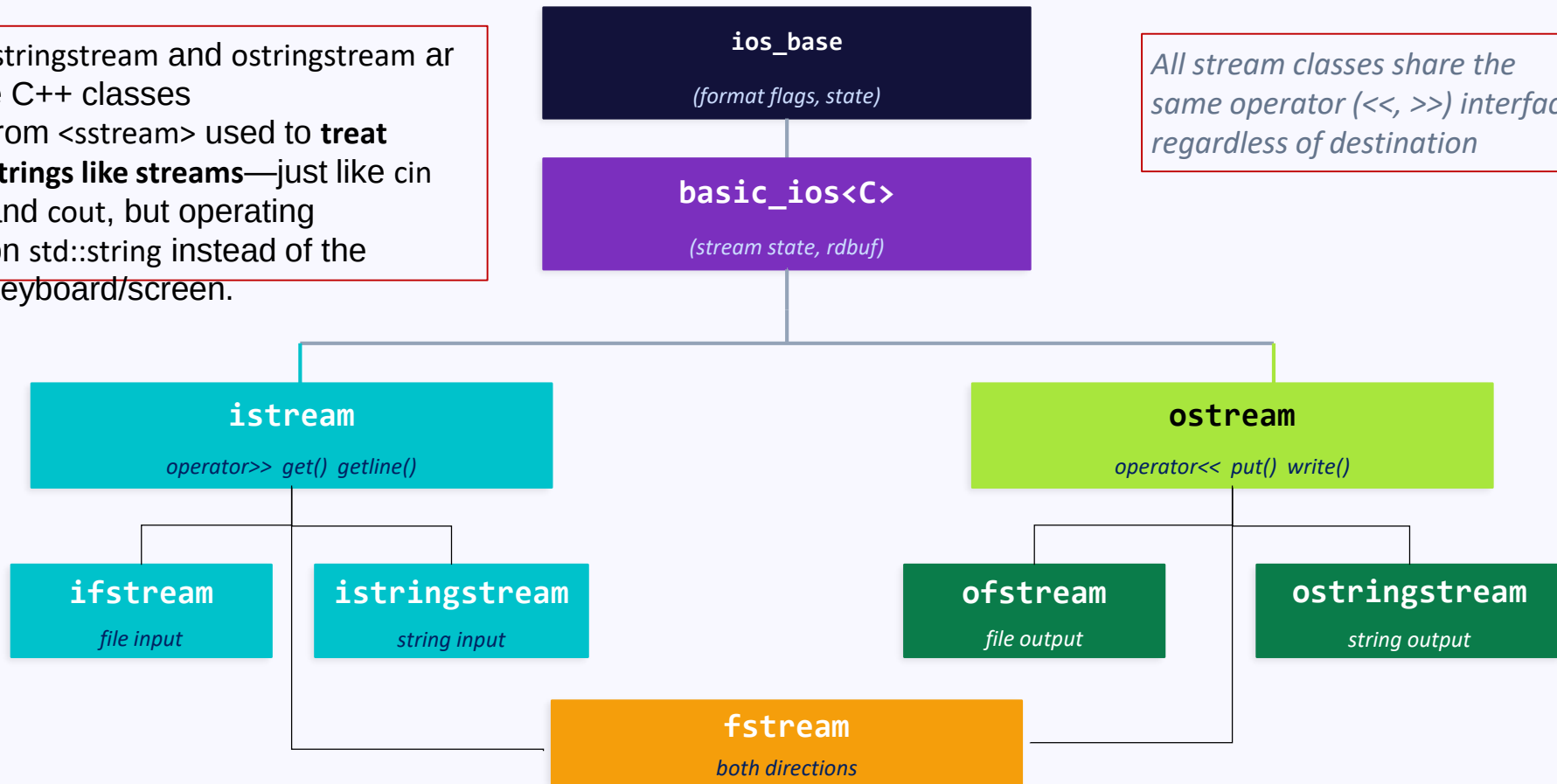
What Are Object Streams?

The C++ stream model · I/O hierarchy · stream as universal interface

C++ STREAM CLASS HIERARCHY

istringstream and ostringstream are C++ classes from `<sstream>` used to **treat strings like streams**—just like `cin` and `cout`, but operating on `std::string` instead of the keyboard/screen.

All stream classes share the same operator (`<<`, `>>`) interface regardless of destination



Object Streams

Stream

- A stream is a sequence of bytes that represents a source or destination of data
- A stream object is an object that either delivers data to its destination (screen, file, etc.) or that takes data from a source (keyboard, file, etc.)
- It acts as a buffer between the data source and destination

Input stream:

- A stream that provides input to a program
- `cin` is an input stream object (`istream` class)

Output stream:

- A stream that accepts output from a program
- `cout` is an output stream object (`ostream` class)

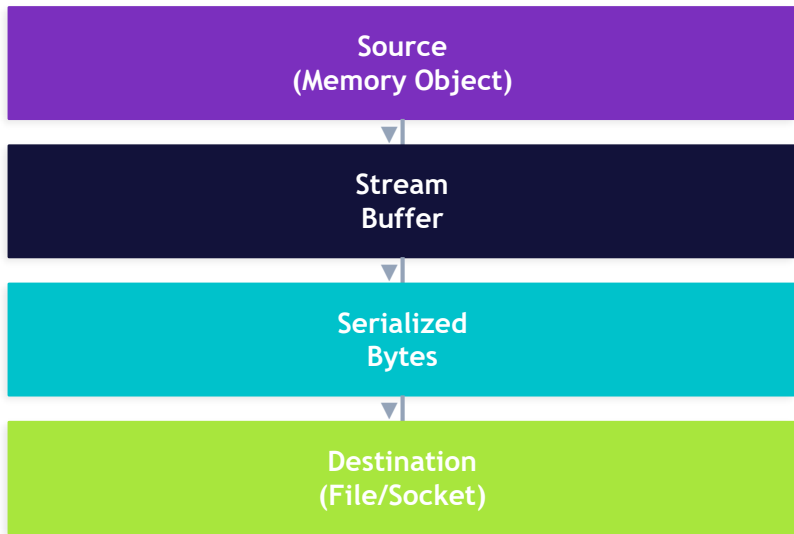
- The `iostream` library in C++ provides several classes for working with input and output streams, including:
 - `istream`: Input stream class used for reading data from a stream
 - `ostream`: Output stream class used for writing data to a stream
 - `iostream`: Input/output stream class that combines the functionalities of `istream` and `ostream`
- These stream classes can be used with various types of stream buffers, responsible for managing the actual data input or output
- Stream buffers can be associated with different types of data sources or destinations, such as files, standard input/output (stdin/stdout), or network sockets

Object Streams

- In C++, file handling can be performed using the **fstream** library, which provides classes for working with files
- The **fstream** library provides three different classes for file handling
 - **ifstream**: Input file stream, used for reading data from a file
 - **ofstream**: Output file stream, used for writing data to a file
 - **fstream**: A combination of **ifstream** and **ofstream**, used for both reading and writing data to a file

WHAT IS A STREAM? THE MENTAL MODEL

❏ Think: A Stream of Water



Stream = abstract channel between a SOURCE and a SINK

Reading: deserialize

Writing: serialize

Key Properties of C++ Streams

Unified interface

Operator << and >> work the same for cout, files, strings, and your custom classes

Buffered I/O

Data staged in a streambuf; flushed on endl, flush(), or destructor call

Type-aware

Overloaded operators handle each type differently

Stateful

Format flags (hex, fixed, setw) persist in the stream object until changed

Exception-capable

exceptions() enables throwing on failbit, badbit, eofbit for RAII error handling

Extensible

Operators << & >> can be defined for any user-defined class

SECTION 02

operator<< and operator>>

Custom stream insertion & extraction · Non-intrusive design

OVERLOADING operator<< AND operator>>

STREAM OPERATOR SYNTAX

```
// — Class definition —————
class Point {
public:
    double x, y, z;
    Point(double x=0, double y=0, double z=0) :
        x(x), y(y), z(z) {}
};
```

```
// — operator<< (insertion - write TO stream) —————
// Must return ostream& so chains work: cout << p1 << " " << p2 << endl;
// Must be non-member (friend) because left operand is ostream, not Point

std::ostream& operator<<(std::ostream& os, const Point& p) {
    os << "(" << p.x << ", " << p.y << ", " << p.z << ")";
    return os;    // ← return the stream to enable chaining
}
```

```
// — operator>> (extraction - read FROM stream) —————
// Takes Point by non-const reference to fill in the fields

std::istream& operator>>(std::istream& is, Point& p) {
    char discard;    // for '(' ',' ' ' ')'
    is >> discard >> p.x >> discard    // read ( x ,
    >> p.y >> discard    // read y ,
    >> p.z >> discard;    // read z )
    return is;    // ← return the stream (enables loop: while(is >> p) {...})
}
```

```
// — Works with ANY stream —————
Point p{1.5, 2.7, 3.14};
cout << p << "\n";    // console: (1.5, 2.7, 3.14)
ofstream f ("points.txt"); f << p;    // file
ostream oss; oss << p;    // string
```

```
// ALL use the same operator <<
```

WHY friend? WHY NON-MEMBER? — THE DESIGN RULES

The Three Rules

1

Left operand is ostream/istream

So it CANNOT be a member of your class (member functions put YOUR class on the left).
operator << (ostream&, YourClass).

2

Declare as friend

To access private members without getters. Alternatively, write getters/setters and make it non-friend.

3

Return the stream reference

Return ostream& / istream& to enable chaining:
cout << a << b << c;
Without return, each << starts fresh.

TWO PATTERNS

```
// — Pattern 1: friend (access private members) —
class Temperature {
    double celsius_;    // private!
    friend std::ostream& operator<<(std::ostream&, const Temperature&);
    friend std::istream& operator>>(std::istream&, Temperature&);
public:
    Temperature(double c=0) : celsius_(c) {}
};

std::ostream& operator<<(std::ostream& os, const Temperature& t) {
    return os << t.celsius_ << "°C"; // accesses private
}

std::istream& operator>>(std::istream& is, Temperature& t) {
    return is >> t.celsius_;
}

// — Pattern 2: non-friend (uses public interface) —
class Temperature2 {
    double celsius_;
public:
    Temperature2(double c=0) : celsius_(c) {}
    double get() const { return celsius_; }
    void set(double c){ celsius_ = c; }
};

std::ostream& operator<<(std::ostream& os, const Temperature2& t) {
    return os << t.get() << "°C"; // uses getter — no friend needed
}

std::istream& operator>>(std::istream& is, Temperature2& t) {
    double v; is >> v; t.set(v); return is;
}
```

REAL EXAMPLE: STUDENT OBJECT STREAM OPERATORS

STUDENT STREAM OPERATORS

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
using namespace std;

class Student {
    int id_;
    string name_;
    double gpa_;
    int year_;
public:
    Student(int id=0, string n="", double g=0.0, int y=1)
        : id_(id), name_(n), gpa_(g), year_(y) {}

    // — Stream insertion (writing) —
    friend ostream& operator<<(ostream& os, const Student& s) {
        return os << s.id_ << "|" << s.name_ << "|"
            << fixed << setprecision(2) << s.gpa_
            << "|" << s.year_;
    }

    // — Stream extraction (reading) —
    friend istream& operator>>(istream& is, Student& s) {
        char pipe;
        return is >> s.id_ >> pipe >> s.name_ >> pipe
            >> s.gpa_ >> pipe >> s.year_;
    }

    void print() const {
        cout << "ID:" << id_ << " Name:" << name_
            << " GPA:" << gpa_ << " Year:" << year_ << "\n";
    }
};

int main() {
    // Write students to file using operator<<
    vector<Student> students {{1,"Ali",3.8,2}, {2,"Sara",3.9,3},
                               {3,"Zain",3.5,4}};

    ofstream out("students.txt");
    for (const auto& s : students) out << s << "\n";

    // Read them back using operator>>
    ifstream in("students.txt");
    Student s;
    while (in >> s) s.print(); //loop ends when >> returns falsy stream
    // Output: ID:1 Name:Ali GPA:3.80 Year:2 ...
}
```

SECTION 03

Stream Flags & Formatting

ios flags · manipulators · format state · precision · width

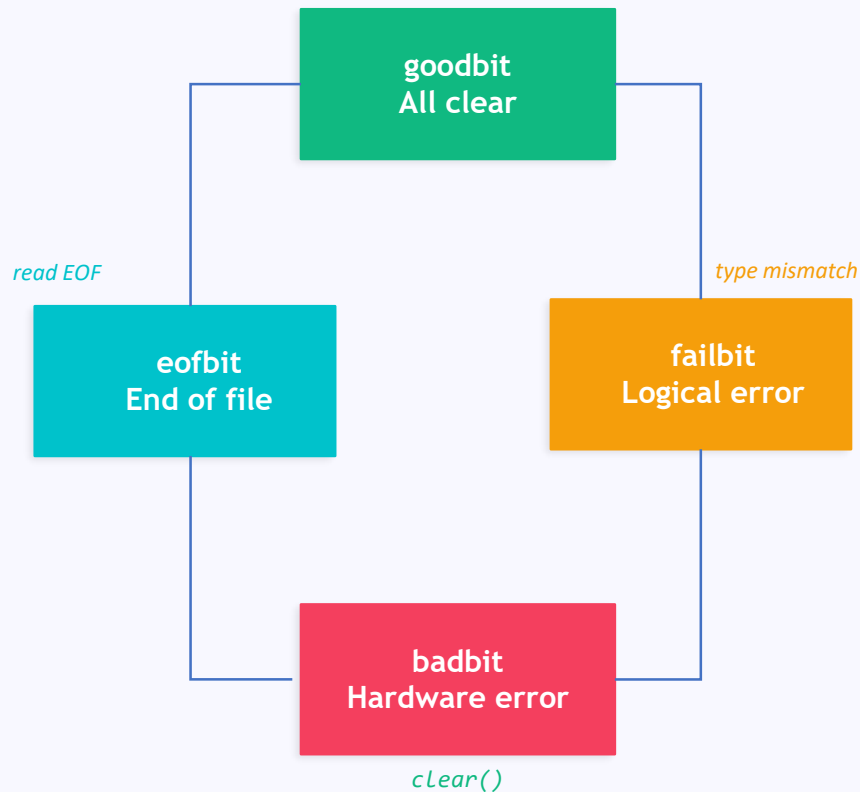
STREAM FLAGS, MANIPULATORS & FORMAT STATE

FLAGS & MANIPULATORS

```
#include <iostream>
#include <iomanip>
using namespace std;
// — Number formatting —
double pi = 3.14159265;
cout << fixed << setprecision(4) << pi << "\n"; // 3.1416 (fixed decimal)
cout << scientific << setprecision(2) << pi << "\n"; // 3.14e+00 (scientific)
cout << defaultfloat << pi << "\n"; // 3.14159 (default)
cout << hexfloat << pi << "\n"; // 0x1.921f...p+1
// — Width, fill, alignment —
cout << setw(12) << left << "Name" << "|"; // "Name      |"
cout << setw(12) << right << 42 << "|"; // "              42|"
cout << setw(12) << setfill('0') << 7 << "|"; // "000000000007|"
cout << setfill(' '); // reset fill char
// — Base (hex/oct/dec) —
int n = 255;
cout << dec << n << " " << hex << n << " " << oct << n << "\n"; // Output: 255 ff 377
cout << showbase << hex << n << "\n"; // 0xff (show prefix)
cout << uppercase << hex << n << "\n"; // 0XFF
// — Boolean —
cout << boolalpha << true << " " << false << "\n"; // true false
cout << noboolalpha << true << " " << false << "\n"; // 1 0

// — Sticky vs non-sticky —
// setw() is NOT sticky – resets after one output
// fixed, hex, left, setfill, setprecision ARE sticky – persist
cout << fixed << setprecision(3); // persists for all subsequent doubles
cout << 1.0 << " " << 2.0 << " " << 3.0; // 1.000 2.000 3.000
```

STREAM STATE MACHINE — good, fail, bad, eof



STREAM STATE

```
// — Check stream state —————
ifstream fin("data.txt");
int n;
fin >> n;

if (fin.good())    cout << "OK: " << n << "\n";
if (fin.eof())     cout << "End of file\n";
if (fin.fail())    cout << "Format error\n";
if (fin.bad())     cout << "Hardware/fatal error\n";
// Compact check – stream is false when failed:
if (!fin)          cout << "Failed!\n";
if (fin)           cout << "OK!\n";

// — Reset state and retry —————
fin.clear();        // clear all error bits → goodbit
fin.seekg(0);       // rewind to beginning

// — Exception mode (RAII) —————
fin.exceptions(ios::failbit | ios::badbit);
try {
    fin >> n;        // throws ios_base::failure on error
}
catch (const ios_base::failure& e) {
    cerr << "Stream error: " << e.what() << "\n";
}
```

SECTION 04

File Streams for Objects

fstream · binary vs text · seekg/seekp · random access

FILE STREAMS WITH OBJECTS — TEXT MODE

FILE STREAMS — TEXT MODE

```
#include <fstream>
#include <vector>
using namespace std;

// Student class with stream operators (from slide 11)

void saveAll(const vector<Student>& v, const string& fname){
    ofstream out(fname);           // text mode by default
    if (!out) throw runtime_error("Cannot open: " + fname);
    out << v.size() << "\n";       // write count first
    for (const auto& s : v) out << s << "\n";
    cout << "Saved " << v.size() << "students to" << fname << "\n";
}

vector<Student> loadAll(const string& fname){
    ifstream in(fname);
    if (!in) throw runtime_error("Cannot open: " + fname);
    int n; in >> n;                 // read count
    vector<Student> result(n);
    for (auto& s : result) in >> s; // use operator>>
    cout << "Loaded " << n << " students\n";
    return result;
}

void appendStudent(const Student& s, const string& fname){
    ofstream out(fname, ios::app); // append – never truncates
    out << s << "\n";
}

int main(){
    vector<Student> roster{{101,"Ali",3.8,2},
                           {102,"Sara",3.9,3}};
    saveAll(roster, "roster.txt");

    appendStudent({103,"Zain",3.5,4}, "roster.txt");

    auto loaded = loadAll("roster.txt");
    for (const auto& s : loaded) cout << s << "\n";

    // — roster.txt —————
    // 2
    // 101|Ali|3.80|2
    // 102|Sara|3.90|3
    // 103|Zain|3.50|4      ← appended
}
```


RANDOM ACCESS — seekg, seekp, tellg, tellp

File as Random-Access Array (binary mode only — fixed-size records for O(1) seek):

seekg →

seekp →

Rec 0
(62B)

Rec 1
(62B)

Rec 2
(62B)

Rec 3
(62B)

Rec 4
(62B)

Rec N
(62B)

RANDOM ACCESS BINARY

```
// — Seek to record n directly — O(1) —————  
struct StudentRecord { uint32_t id; char name[50]; float gpa;  
int year; };  
const string BIN = "students.bin";
```

```
Student readRecord(int n) {  
    ifstream fin(BIN, ios::binary);  
    fin.seekg(n*sizeof(StudentRecord)); //jump to exact byte offset  
    StudentRecord rec;  
    fin.read(reinterpret_cast<char*>(&rec), sizeof(rec));  
    return {rec.id, rec.name, rec.gpa, rec.year};  
}
```

```
void updateGPA(int n, float newGPA) {  
    fstream f(BIN, ios::binary | ios::in | ios::out);
```

```
streampos pos = n * sizeof(StudentRecord);  
f.seekg(pos); // read position  
StudentRecord rec;  
f.read(reinterpret_cast<char*>(&rec), sizeof(rec));  
rec.gpa = newGPA;  
f.seekp(pos); // write position (back to same record!)  
f.write(reinterpret_cast<char*>(&rec), sizeof(rec));
```

```
int totalRecords() {  
    ifstream fin(BIN, ios::binary | ios::ate); // open at end  
    return fin.tellg() / sizeof(StudentRecord);  
    // total size / record size = count  
}
```

SECTION 05

String Streams (sstream)

ostreamstream · istreamstream · round-trip

STRING STREAMS — IN-MEMORY SERIALIZATION BUFFERS

STRING STREAMS (sstream)

```
#include <sstream>
using namespace std;
// — ostream: object → string (serialize to memory) —
Student s{101, "Ali Khan", 3.8, 2};
ostream oss;
oss << s; // uses operator<<
string serialized = oss.str(); // "101|Ali Khan|3.80|2"
cout << "Serialized: [" << serialized << "]\n";

// — istream: string → object (deserialize from memory) —
string raw = "102|Sara Baig|3.90|3";
istream iss(raw);
Student loaded;
iss >> loaded; // uses operator>>
loaded.print(); // ID:102 Name:Sara Baig GPA:3.90

// — Round-trip test —
auto roundTrip = [](const Student& in) -> Student {
    ostream out_ss; out_ss << in; // object → string
    istream in_ss(out_ss.str());
    Student out; in_ss >> out; // string → object
    return out;
};

Student original{103, "Zain Mir", 3.5, 4};
Student copy = roundTrip(original); // serialize then deserialize
// copy should equal original — great for unit testing!

// — Use case: send objects over network —
// 1. Serialize all students to one string
ostream packet;
for (const auto& student : roster)
    packet << student << "\n";
string networkPayload = packet.str();
// ready to send via socket / REST

// 2. Receive and reconstruct
istream recv(networkPayload);
Student tmp;
while (recv >> tmp) roster.push_back(tmp);
```

GENERIC SERIALIZE / DESERIALIZE VIA STRING STREAMS

GENERIC STREAM SERIALIZATION

//Generic serialize function – works for ANY type with operator<<

```
template<typename T>
std::string serialize(const T& obj) {
    std::ostringstream oss;
    oss << obj;          // delegates to T's operator<<
    return oss.str();
}
```

//Generic deserialize function – works for ANY type with operator>>

```
template<typename T>
T deserialize(const std::string& s) {
    T obj;
    std::istringstream iss(s);
    iss >> obj;          // delegates to T's operator>>
    if (iss.fail()) throw std::invalid_argument("Deserialize failed: "
s);
    return obj;
}
```

// — Works for built-in types too —————

```
std::string sInt  = serialize(42);          // "42"
std::string sDb1  = serialize(3.14);        // "3.14"
std::string sBool = serialize(true);        // "1"
```

```
int    i = deserialize<int>    ("42");      // 42
double d = deserialize<double>("3.14");     // 3.14
```

// — Works for custom types with stream operators —————

```
std::string sStudent = serialize(Student{101,"Ali",3.8,2});
Student back = deserialize<Student>(sStudent);
```

// — Collection serialization —————

```
template<typename T>
std::string serializeVec(const std::vector<T>& v) {
    std::ostringstream oss;
    oss << v.size() << "\n";
    for (const auto& item : v) oss << item << "\n";
    return oss.str();
}
```

```
+template<typename T>
std::vector<T> deserializeVec(const std::string& s) {
    std::istringstream iss(s);
    int n; iss >> n;
    std::vector<T> result(n);
    for (auto& item : result) iss >> item;
    return result;
}
```

SECTION 06

Real-World Projects

PROJECT 1: GAME SAVE SYSTEM USING OBJECT STREAMS

GAME SAVE – OBJECT STREAMS

```
#include <fstream>
#include <vector>
#include <sstream>
using namespace std;

struct Vec3 { float x,y,z; };
struct Item { string name; int count; int level; };
struct Quest { string id; bool completed; int progress; };

class GameState {
    string  playerName_;
    int     level_, health_, score_;
    Vec3    position_;
    vector<Item> inventory_;
    vector<Quest> quests_;

    // — Stream operators for Vec3, Item, Quest —
    friend ostream& operator<<(ostream& os, const Vec3& v) {
        return os << v.x << " " << v.y << " " << v.z;
    }
    friend istream& operator>>(istream& is, Vec3& v) {
        return is >> v.x >> v.y >> v.z;
    }
    friend ostream& operator<<(ostream& os, const Item& it) {
        return os << it.name << "|" << it.count << "|" << it.level;
    }
    friend istream& operator>>(istream& is, Item& it) {
        char p; return is >> it.name >> p >> it.count >> p >> it.level;
    }
};
```

```
// — GameState stream operators —
friend ostream& operator<<(ostream& os, const GameState& gs) {
    os << gs.playerName_ << "\n" << gs.level_ << " " << gs.health_
        << " " << gs.score_ << "\n" << gs.position_ << "\n";
    os << gs.inventory_.size() << "\n";
    for (auto& it : gs.inventory_) os << it << "\n";
    os << gs.quests_.size() << "\n";
    for (auto& q : gs.quests_)
        os << q.id << " " << q.completed << " " << q.progress << "\n";
    return os;
}
friend istream& operator>>(istream& is, GameState& gs) {
    is >> gs.playerName_ >> gs.level_ >> gs.health_ >> gs.score_ >> gs.position_;
    int n; is >> n; gs.inventory_.resize(n);
    for (auto& it : gs.inventory_) is >> it;
    is >> n; gs.quests_.resize(n);
    for (auto& q : gs.quests_) is >> q.id >> q.completed >> q.progress;
    return is;
}

public:
    void save(int slot) const {
        ofstream f("save" + to_string(slot) + ".sav");
        f << *this;  cout << "Game saved to slot " << slot << "\n";
    }
    void load(int slot) {
        ifstream f("save" + to_string(slot) + ".sav");
        if (!f) throw runtime_error("No save in slot " + to_string(slot));
        f >> *this;  cout << "Loaded: " << playerName_ << " Level " << level_ << "\n";
    }
};
```

PROJECT 2: LAYERED CONFIG WITH STRING STREAMS

CONFIG SYSTEM

```
#include <sstream>
#include <map>
#include <fstream>
using namespace std;

class TypedConfig {
    map<string,string> data_;

    // — Save all key-value pairs to stream

    friend ostream& operator<<(ostream& os, const TypedConfig& cfg) {
        for (const auto& [k,v] : cfg.data_) os << k << " = " << v <<
"\n";
        return os;
    }
    // — Load key-value pairs from stream

    friend istream& operator>>(istream& is, TypedConfig& cfg) {
        string line;
        while (getline(is, line)) {
            if (line.empty() || line[0]=='#') continue;
            auto eq = line.find(" = ");
            if (eq != string::npos)
                cfg.data_[line.substr(0,eq)] = line.substr(eq+3);
        }
        return is;
    }
public:
    // Type-safe accessors
    template<typename T>
    void set(const string& key, const T& val) {
        ostringstream oss; oss << val;    // use operator<< to convert
        data_[key] = oss.str();
    }
    template<typename T>
    T get(const string& key, T def={}) const {
        auto it = data_.find(key);
        if (it == data_.end()) return def;
        T result; istringstream(it->second) >> result; // use operator>>
        return result;
    }
    void loadFile(const string& file) { ifstream f(file); f >> *this; }
    void saveFile(const string& file) const { ofstream f(file); f <<
*this; }
    // Merge from another config (environment overrides)
    void merge(const TypedConfig& other) {
        for (auto& [k,v] : other.data_) data_[k] = v;
    }
};

int main() {
    TypedConfig cfg;
    cfg.set("host", string("localhost"));
    cfg.set("port", 8080);
    cfg.set("debug", true);
    cfg.saveFile("app.cfg");

    TypedConfig loaded; loaded.loadFile("app.cfg");
    cout << loaded.get<string>("host","") << "\n"; // localhost
    cout << loaded.get<int> ("port", 0) << "\n"; // 8080
    cout << loaded.get<bool> ("debug",0) << "\n"; // 1
}
```

PROJECT 3: NETWORK OBJECT SERIALIZATION FRAMEWORK

NETWORK SERIALIZATION

```
// — Serialize objects to/from string for network transport

#include <sstream>
#include <memory>
using namespace std;

// Any object with operator<< / operator>> can be sent over a network!
class NetworkSerializer {
public:
    // Serialize: object → wire string
    template<typename T>
    static string toWire(const T& obj, int msgType) {
        ostringstream payload;
        payload << obj;
        ostringstream frame;
        frame << msgType << "|" << payload.str().size() << "|" <<
        payload.str();
        return frame.str();
    }

    // Deserialize: wire string → object
    template<typename T>
    static T fromWire(const string& frame, int& outMsgType) {
        istream iss(frame);
        size_t payloadLen;
        char pipe;
        iss >> outMsgType >> pipe >> payloadLen >> pipe;
        string payload(payloadLen, ' ');
        iss.read(&payload[0], payloadLen);
        T obj; istream(payload) >> obj;
    }
};
```

```
        return obj;
    }
};

// — Message types

enum MsgType { LOGIN=1, CHAT=2, STUDENT_UPDATE=3 };

struct ChatMessage { string from, text; uint64_t ts; };
ostream& operator<<(ostream& os, const ChatMessage& m){return
os<<m.from<<"|"<<m.text<<"|"<<m.ts;}
istream& operator>>(istream& is, ChatMessage& m){char p;return
is>>m.from>>p>>m.text>>p>>m.ts;}

int main() {
    ChatMessage msg{"Alice","Hello network!",1705012345ULL};

    // Serialize to wire format
    string wire = NetworkSerializer::toWire(msg, CHAT);
    cout << "Wire: [" << wire << "]\n";    // 2|28|Alice|Hello
    network!|1705012345

    // Deserialize on receiver side
    int type;
    ChatMessage recv = NetworkSerializer::fromWire<ChatMessage>(wire,
    type);
    cout << "From: " << recv.from << " Msg: " << recv.text << "\n";
}
```


PROJECT 4: TYPE-SAFE LOGGER USING OBJECT STREAMS

TYPE-SAFE LOGGER

```
#include <sstream>
#include <fstream>
#include <ctime>
using namespace std;

enum class LogLevel { DEBUG, INFO, WARN, ERROR, CRITICAL };

class Logger {
    ofstream file_;
    LogLevel minLevel_;
    string levelTag(LogLevel lv){
        switch(lv){case LogLevel::DEBUG:return"[DBG]";case
LogLevel::INFO:return"[INF]";
                case LogLevel::WARN:return"[WRN]";case
LogLevel::ERROR:return"[ERR]";
                default:return"[CRT]";}
    }
    string timestamp(){
        time_t now=time(nullptr);char buf[24];
        strftime(buf,sizeof(buf),"%Y-%m-%d
%H:%M:%S",localtime(&now));return buf;
    }
public:
    Logger(const string& path, LogLevel min=LogLevel::INFO
        : file_(path, ios::app), minLevel_(min) {}

    // — Core: accepts ANY loggable object that has operator<<

    template<typename... Args>
    void log(LogLevel lv, const Args&... args) {

        if (lv < minLevel_) return;
        ostringstream msg;
        msg << timestamp() << " " << levelTag(lv) << " ";
        (msg << ... << args);          // C++17 fold: apply << to each
        arg

        string line = msg.str();
        cout << line << "\n";
        file_ << line << "\n";
        file_.flush();
    }

    // Convenience helpers
    template<typename... A> void info    (A&&... a){log(LogLevel::INFO,
a...);}
    template<typename... A> void warn   (A&&... a){log(LogLevel::WARN,
a...);}
    template<typename... A> void error  (A&&... a){log(LogLevel::ERROR,
a...);}
};

int main() {
    Logger logger("app.log", LogLevel::INFO);
    Student s{101,"Ali",3.8,2};

    // log accepts ANY object with operator<<
    logger.info ("Server started on port ", 8080);
    logger.warn ("Memory at ", 87, "% – Student: ", s);    // Student
uses its operator<<
    logger.error("Connection failed – retrying in ", 5, "s");
}
```

PROJECT 5: OBJECT STREAM PIPELINE (ETL PATTERN)

ETL PIPELINE

```
// — ETL: Extract-Transform-Load using object streams

#include <sstream>
#include <fstream>
#include <vector>
#include <algorithm>
using namespace std;

// EXTRACT: read objects from CSV stream
template<typename T>
vector<T> extract(istream& source) {
    vector<T> data;
    T obj;
    while (source >> obj) data.push_back(obj);
    return data;
}

// TRANSFORM: filter + map — returns new vector
template<typename T, typename Pred, typename Mapper>
auto transform_data(const vector<T>& data, Pred predicate, Mapper mapper)
{
    using Out = decltype(mapper(data[0]));
    vector<Out> result;
    for (const auto& item : data)
        if (predicate(item)) result.push_back(mapper(item));
    return result;
}

// LOAD: write objects to output stream
template<typename T>
void load(ostream& dest, const vector<T>& data) {
    for (const auto& item : data) dest << item << "\n";
}

int main() {
    // EXTRACT: read all students from file
    ifstream csvIn("students.csv");
    auto students = extract<Student>(csvIn);

    // TRANSFORM: keep only GPA >= 3.7, convert to report string
    auto topStudents = transform_data(students,
        [](const Student& s){ return s.gpa() >= 3.7; }, // predicate
        [](const Student& s){ return s; });             // identity
    mapper

    // LOAD to console + file simultaneously using tee-stream pattern
    ofstream report("report.txt");
    load(cout, topStudents); // to console
    load(report, topStudents); // to file

    // LOAD stats to ostringstream, then send as email body
    ostringstream summary;
    summary << "Top students: " << topStudents.size() << " of " <<
students.size() << "\n";
    load(summary, topStudents);
    string emailBody = summary.str();
    cout << "\nEmail body:\n" << emailBody;
}
```

SECTION 07

Best Practices & Summary

BEST PRACTICES FOR OBJECT STREAM SERIALIZATION

✓ Return the stream reference

Always return ostream& / istream& from operators. Enables chaining: cout << a << b;

✓ Declare operator<< as friend

Gives access to private members without breaking encapsulation through getters.

✓ Test round-trip fidelity

serialize → deserialize must produce identical object. Unit-test edge cases (empty string, NaN, INT_MAX).

✓ Use setprecision for floats

Text streams lose precision by default. Use setprecision(numeric_limits<double>::digits10+1) for lossless float.

✓ Use delimiters carefully

Choose delimiter that cannot appear in data (pipe | for names with spaces; or use length-prefix for binary).

⚠ Whitespace issues in >>

operator>> skips leading whitespace by default. Use getline() for fields with spaces.

⚠ setw() is NOT sticky

Width resets after each output. Set it before EVERY field that needs alignment.

✗ Never trust stream input

Validate all deserialized data. Malformed input can cause integer overflow, OOM, or crashes.

✗ Don't mix binary/text modes

Opening a binary file in text mode corrupts data on Windows (\n↔\r\n translation).

✗ Don't ignore stream state

Always check !stream or stream.good() after reads. Silent failures propagate silently downstream.

CAPSTONE: COMPLETE RECORD STORE WITH OBJECT STREAMS

CAPSTONE RECORD STORE

```
// Combines: operator<</>>, fstream, sstream, validation
#include <fstream>
#include <sstream>
#include <vector>
#include <stdexcept>
using namespace std;

class Product {
    uint32_t id_; string name_; float price_; int stock_;
    friend ostream& operator<<(ostream& os, const Product& p){
        return os << p.id_ << "|" << p.name_ << "|"
            << fixed << setprecision(2) << p.price_ << "|" << p.stock_;
    }
    friend istream& operator>>(istream& is, Product& p){
        char pipe;
        if(!(is >> p.id_ >> pipe)) return is;    // check each step
        if(!getline(is, p.name_, '|')) return is; //name have spaces
        is >> p.price_ >> pipe >> p.stock_;
        // Validate
        if(p.price_ < 0 || p.stock_ < 0) { is.setstate(ios::failbit);
    }
        if(p.name_.empty()) { is.setstate(ios::failbit); }
        return is;
    }
public:
    Product(uint32_t id=0, string n="", float p=0, int
s=0):id_(id),name_(n),price_(p),stock_(s){}
```

```
// — RecordStore wraps file persistence —————
class RecordStore {
    string path_;
public:
    RecordStore(string p):path_(p){}
    void saveAll(const vector<Product>& v){
        ofstream f(path_); f << v.size() << "\n";
        for(auto& p:v) f << p << "\n";
    }
    vector<Product> loadAll(){
        ifstream f(path_); if(!f) return {};
        int n; f >> n; f.ignore();
        vector<Product> result; Product p;
        while(n-- > 0 && f >> p) result.push_back(p);
        return result;
    }
    // In-memory search using istringstream
    vector<Product> search(const string& keyword){
        auto all = loadAll();
        vector<Product> found;
        for(auto& p : all){
            ostringstream oss; oss << p;    // serialize to string
            if(oss.str().find(keyword) != string::npos) found.push_back(p);
        }
        return found;
    }
};
```

OBJECT STREAMS — QUICK REFERENCE CHEAT SHEET

CHEAT SHEET

```
// — operator<< / operator>> skeleton —————
class MyClass {
    int x_; string s_;
    friend ostream& operator<<(ostream& os, const MyClass& obj)
        { return os << obj.x_ << "|" << obj.s_; }
    friend istream& operator>>(istream& is, MyClass& obj) {
        char p; return is >> obj.x_ >> p >> obj.s_; }
};

// — Works with ALL stream destinations —————
MyClass obj{42,"hello"};
cout << obj; // console
ofstream("out.txt") << obj; // file (text)
ofstream("out.bin",ios::binary) << obj; // file (binary)
ostringstream oss; oss << obj; // string: oss.str()
// string → object
MyClass copy; istringstream(oss.str()) >> copy;

// — Stream flags (sticky unless stated) —————
cout << fixed << setprecision(6); // sticky floating-
point format
cout << setw(12); // NOT sticky —
resets each output
cout << left << setfill('0'); // sticky alignment + fill
```

```
// — Stream state —————
if (!stream) // failed?
stream.clear(); // reset all error bits
stream.ignore(); // skip one char (e.g., leftover newline)
stream.ignore(1000, '\n'); // skip until newline
```

COMMON MISTAKES & HOW TO FIX THEM

1. operator<< as member function

✗ WRONG

```
class Point {
public:
    // WRONG: member means: p << cout
    ostream& operator<<(ostream& os){
        return os << x << " " << y;
    }
};
```

✓ FIX

```
class Point {
    // RIGHT: non-member/friend
    friend ostream& operator<<(
        ostream& os, const Point& p){
        return os << p.x << " " << p.y;
    }
};
```

2. Forgetting to return stream

✗ WRONG

```
ostream& operator<<(ostream& os,
const T& t){
    os << t.x;
    // WRONG: forgot return!
} // compiler error
```

✓ FIX

```
ostream& operator<<(ostream& os, const
T& t){
    os << t.x;
    return os; // RIGHT: enable chaining
} // cout << a << b << c now works
```

3. Float precision loss

✗ WRONG

```
ofstream f("data.txt");
f << 3.14159265358979;
//writes 3.14159

// When loaded back: only 6 sig
digits!
```

✓ FIX

```
ofstream f("data.txt");
f << fixed << setprecision(17)
    << 3.14159265358979; // all digits
// Or:
numeric_limits<double>::digits10+1
```

4. Mixing getline and >>

✗ WRONG

```
string name; int age;
cin >> age;
getline(cin, name);
// reads the leftover \n!

// name is empty string - bug!
```

✓ FIX

```
string name; int age;
cin >> age;
cin.ignore(); // eat leftover \n
getline(cin, name);
// now reads real name
```

THANK YOU

What you've mastered:

● Stream Hierarchy

● `operator<</>>`

● Stream Flags

● `fstream + seekg`

● `stringstream` buffers

● Boost Archives

● Object Graphs

● Polymorphism

● Versioning

● ETL Pipeline

● Design Patterns

● Best Practices